

PETER-PROJECT

車牌辨識系統 教學手冊

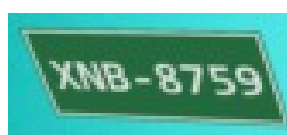
從合成資料、CRNN + CTC 模型、訓練流程，到 Flask 網站的程式碼導讀



999-PRW 000000.jpg



RR-9702 000001.jpg



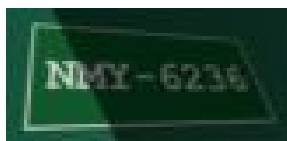
XNB-8759 000002.jpg



HAV-7819 000003.jpg



8016-VM 000004.jpg



NMY-6236 000005.jpg



1242-LX 000006.jpg



9964-NZ 000007.jpg

以上是專案自己產生的驗證集圖片 (dataset/val)，底下是正確答案與檔名。

文件產生日期：2026-10-02

目錄

1. 這個專案在做什麼	3
檔案一覽	3
2. 快速開始	4
3. 合成資料集：generate_dataset.py	4
3.1 字元集	5
3.2 車牌格式與顏色	5
3.3 畫出乾淨的車牌：render_plate()	6
3.4 資料增強：augment()	6
3.5 多行程平行產生	9
4. 模型：plate_model.py	9
4.1 影像前處理與文字編碼	9
4.2 CRNN 的結構	10
5. CTC：不用標位置也能學會讀字	11
5.1 推論：greedy_decode()	11
6. 訓練流程：train.py	12
6.1 準備	12
6.2 訓練迴圈	12
6.3 驗證：兩種正確率	13
6.4 你的真實訓練結果	14
6.5 存檔與訓練進度	14
7. 網站：app.py 與前端	15
7.1 API 一覽	15
7.2 模型熱更新	16
7.3 從網頁啟動訓練	16
7.4 上傳與框選	17
8. 動手實驗	17
目前的限制	17
常見問題	17
附錄：名詞對照	18

1. 這個專案在做什麼

這個專案用 PyTorch 從零開始訓練一個「車牌文字辨識」模型：給它一張已經裁切好的車牌照，它會讀出上面的文字，例如 **ABC-1234**。整個流程不需要下載任何資料集——訓練資料是程式自己畫出來的合成車牌；訓練好的模型則透過一個 Flask 網站展示，可以上傳照片、隨機測試、看訓練曲線，甚至直接在網頁上按按鈕訓練。

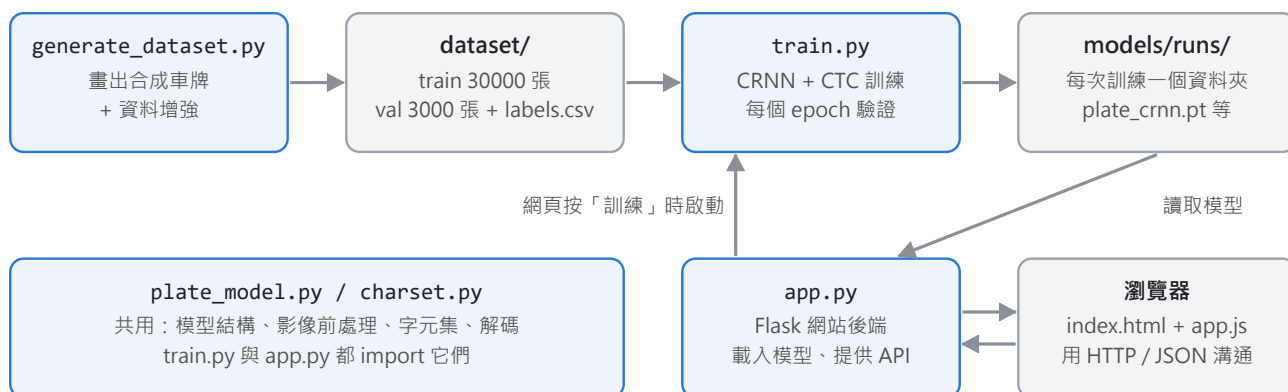


圖 1 整體架構：藍框是程式，灰框是資料或畫面。

讀完這份手冊，你會了解：

- 怎麼用程式產生大量、帶有正確答案的訓練圖片，以及資料增強為什麼重要
- CRNN (CNN + 雙向 LSTM) 怎麼把一張圖變成一串字元的機率
- CTC Loss 如何讓模型在不知道每個字確切位置的情況下學會讀字
- 一個完整的 PyTorch 訓練迴圈：資料、loss、optimizer、learning rate 排程、驗證、存檔
- 怎麼把模型包成網站，並讓網頁能啟動訓練、即時顯示進度

檔案一覽

檔案	行數	負責什麼
charset.py	2	字元集：0-9、A-Z (不含 I、O) 和 -，共 35 個字元
generate_dataset.py	249	畫出合成車牌、做資料增強，輸出 dataset/ 的圖片與 labels.csv
plate_model.py	73	模型結構 (CRNN)、影像前處理、文字編碼與解碼、載入模型
train.py	227	訓練流程：讀資料、訓練、驗證、存最佳模型、寫訓練紀錄與進度
app.py	300	Flask 網站後端：辨識 API、模型管理、從網頁啟動產生資料 / 訓練
templates/index.html	113	網頁的版面
static/app.js	383	網頁的互動：框選車牌、呼叫 API、畫訓練曲線與進度條
static/style.css	98	網頁樣式

2. 快速開始

所有指令都在專案資料夾 **D:\Peter Project** 裡執行。

終端機

```
pip install -r requirements.txt      # flask、torch、numpy、Pillow

# 1. 產生資料集 (預設 train 30000 張、val 3000 張；已存在會自動略過)
python generate_dataset.py

# 2. 訓練模型 (每次訓練存成 models/runs/<日期-時間>-<名稱>/)
python train.py --epochs 30 --name baseline

# 3. 開網站，然後用瀏覽器打開 http://127.0.0.1:5000
python app.py
```

注意 不能直接雙擊 **templates/index.html** 打開：它是 Flask 樣板，裡面的 `{{ url_for(...) }}` 要由伺服器換成真正的路徑，而且辨識要靠後端的 PyTorch 執行。一定要先跑 **python app.py**。

步驟 1、2 也可以不打指令：開好網站後，在最上方的「訓練工作台」設定參數按按鈕即可。網站的各區塊：

區塊	可以做什麼
訓練工作台	產生資料集、設定 epochs / batch size / learning rate 後開始訓練；右側有進度條和執行紀錄
已儲存的模型	列出每次訓練的參數和最佳成績，按「使用」切換網站要用的模型
上傳圖片辨識	上傳照片，在圖上拖曳框出車牌再按辨識（模型只會讀字，不會自己找車牌）
隨機產生測試車牌	即時畫一張模型沒看過的車牌來考它
訓練曲線	loss 和驗證正確率隨 epoch 的變化，訓練中會自動更新
驗證集抽樣結果	隨機抽 24 張驗證圖片，用綠框 / 紅框標出對錯

以模型 20261002-000605 為例：用 30000 張圖訓練 5 個 epoch，每個 epoch 約 36 秒，最好的整張正確率達到 **87.8%**、字元正確率 **98.1%**（詳見第 6 章的訓練曲線）。

3. 合成資料集：generate_dataset.py

深度學習需要大量「圖片 + 正確答案」的配對。真實車牌照既難收集、又得一張張人工標註；這個專案改用程式**直接畫出車牌**——因為文字是我們自己決定的，答案天生就是對的，要幾張就有幾張。

3.1 字元集

charset.py

```
# 車牌字元集：台灣車牌不使用 I、O (避免與 1、0 混淆)
CHARS = "0123456789ABCDEFGHJKLMNPQRSTUVWXYZ-"
```

台灣車牌不使用 I 和 O (避免和 1、0 混淆)，所以字元集是 10 個數字 + 24 個英文字母 + -，共 35 個。模型輸出時會再多一類 **blank** (第 5 章會解釋)，所以總共 36 類。

3.2 車牌格式與顏色

generate_dataset.py

```
FORMATS = [
    ("LLL-DDDD", 0.45), # 新式自用小客車
    ("LL-DDDD", 0.15), # 舊式
    ("DDDD-LL", 0.15), # 舊式
    ("LLL-DDD", 0.15), # 機車
    ("DDD-LLL", 0.10),
]

# (底色, 字色) 組合
COLOR_SCHEMES = [
    ((245, 245, 240), (20, 20, 20)), # 白底黑字：自用
    ((245, 245, 240), (190, 30, 30)), # 白底紅字：營業
    ((245, 245, 240), (30, 120, 60)), # 白底綠字：電動車
    ((30, 110, 60), (245, 245, 240)), # 綠底白字：營業大客車
    ((235, 200, 40), (20, 20, 20)), # 黃底黑字
    ((200, 30, 30), (245, 245, 240)), # 紅底白字
]

SCHEME_WEIGHTS = [0.55, 0.15, 0.1, 0.08, 0.07, 0.05]
```

`random_text()` 先依權重抽一種格式，再把 L 換成隨機英文字母、D 換成隨機數字。顏色組合也一樣依權重抽，所以常見的白底黑字最多。

3.3 畫出乾淨的車牌：render_plate()

generate_dataset.py

```
def render_plate(text, fonts):
    """畫出正面、乾淨的車牌。"""
    W, H = 380, 160
    bg, fg = random.choices(COLOR_SCHEMES, weights=SCHEME_WEIGHTS)[0]
    plate = Image.new("RGB", (W, H), bg)
    draw = ImageDraw.Draw(plate)

    # 外框
    border = random.randint(3, 7)
    draw.rounded_rectangle([3, 3, W - 4, H - 4], radius=14, outline=fg, width=border) \
        if hasattr(draw, "rounded_rectangle") else \
        draw.rectangle([3, 3, W - 4, H - 4], outline=fg, width=border)

    # 螺絲孔
    if random.random() < 0.5:
        for x in (W * 0.22, W * 0.78):
            draw.ellipse([x - 5, 12, x + 5, 22], fill=(150, 150, 150))

    # 文字：選字型並縮放到適合大小
    font_path = random.choice(fonts)
    size = int(H * random.uniform(0.62, 0.78))
    font = get_font(font_path, size)
    tw, th = draw.textsize(text, font=font)
    max_w = W * random.uniform(0.82, 0.92)
    if tw > max_w:
        size = int(size * max_w / tw)
        font = get_font(font_path, size)
        tw, th = draw.textsize(text, font=font)
    ox, oy = font.getoffset(text)
    x = (W - tw) / 2 + random.uniform(-6, 6)
    y = (H - th - oy) / 2 + random.uniform(4, 12) # 偏下方，上方留給螺絲孔
    draw.text((x, y - oy / 2), text, font=font, fill=fg)
    return plate
```

在 380×160 的畫布上畫底色、圓角外框、（一半機率）螺絲孔，再從 7 種 Windows 內建字型中隨機挑一種寫字。字太寬時會等比例縮小，位置也加了一點隨機偏移，讓模型不會只認得「永遠在正中央」的字。

3.4 資料增強：augment()

乾淨的車牌和真實照片差很多。`augment()` 依序加上各種「真實世界的髒東西」：

步驟	做法	模擬什麼
透視變形	隨機抖動四個角，用 <code>Image.PERSPECTIVE</code> 變形	從斜角拍攝
隨機背景	隨機底色 + 漸層 + 雜訊	車牌周圍的車身、路面
隨機裁切邊界	四周多留 0–15% 的背景	人工框選不精準
陰影	30% 機率蓋上半透明多邊形	樹影、建築物陰影
亮度 / 對比	各乘上 0.5–1.4 倍	白天、夜晚、逆光

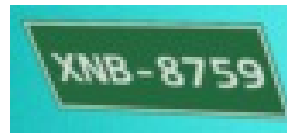
步驟	做法	模擬什麼
模糊	30% 高斯模糊、15% 水平動態模糊	對焦失準、車子在移動
低解析度	縮到寬 70-260 像素	遠距離拍攝
雜訊 + JPEG	50% 加高斯雜訊；存檔品質 35-95	感光元件雜訊、壓縮失真



999-PRW 000000.jpg



RR-9702 000001.jpg



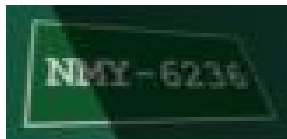
XNB-8759 000002.jpg



HAV-7819 000003.jpg



8016-VM 000004.jpg



NMY-6236 000005.jpg



1242-LX 000006.jpg



9964-NZ 000007.jpg

圖 2 增強後的真實輸出 (dataset/val 前 8 張)。有些已經模糊到人眼都要看一下。

generate_dataset.py

```

def augment(plate):
    """透視變形、背景、模糊、光線、雜訊、低解析度、JPEG 壓縮。"""
    W, H = plate.size
    pad = int(W * 0.35)
    cw, ch = W + 2 * pad, H + 2 * pad
    canvas = random_background(cw, ch)

    # 隨機抖動四個角，做透視變形
    j = 0.12
    src = [(0, 0), (W, 0), (W, H), (0, H)]
    dst = [
        (pad + x + random.uniform(-j, j) * W, pad + y + random.uniform(-j, j) * H)
        for x, y in src
    ]
    coeffs = perspective_coeffs(src, dst)
    warped = plate.transform((cw, ch), Image.PERSPECTIVE, coeffs, Image.BICUBIC)
    mask = Image.new("L", (W, H), 255).transform((cw, ch), Image.PERSPECTIVE, coeffs, Image.BICUBIC)
    canvas.paste(warped, (0, 0), mask)

    # 依車牌四角裁切，四周留隨機邊界
    xs, ys = [p[0] for p in dst], [p[1] for p in dst]
    bw, bh = max(xs) - min(xs), max(ys) - min(ys)
    img = canvas.crop((
        int(min(xs) - random.uniform(0, 0.08) * bw),
        int(min(ys) - random.uniform(0, 0.15) * bh),
        int(max(xs) + random.uniform(0, 0.08) * bw),
        int(max(ys) + random.uniform(0, 0.15) * bh),
    ))

    # 陰影
    if random.random() < 0.3:
        shade = Image.new("L", img.size, 0)
        sw, sh = img.size
        x0 = random.uniform(0, sw)
        ImageDraw.Draw(shade).polygon(
            [(x0, 0), (sw, 0), (sw, sh), (x0 + random.uniform(-sw / 2, sw / 2), sh)],
            fill=random.randint(60, 140),
        )
        img = Image.composite(Image.new("RGB", img.size, (0, 0, 0)), img, shade)

    # 光線、對比
    img = ImageEnhance.Brightness(img).enhance(random.uniform(0.5, 1.4))
    img = ImageEnhance.Contrast(img).enhance(random.uniform(0.5, 1.4))

    # 模糊 (一般 / 動態)
    r = random.random()
    if r < 0.3:
        img = img.filter(ImageFilter.GaussianBlur(random.uniform(0.5, 2.0)))
    elif r < 0.45:
        k = [0] * 10 + [1] * 5 + [0] * 10 # 5x5 水平動態模糊
        img = img.filter(ImageFilter.Kernel((5, 5), k, scale=5))

    # 低解析度 (模擬遠距離拍攝)
    target_w = random.randint(70, 260)
    target_h = max(16, int(img.height * target_w / img.width))
    img = img.resize((target_w, target_h), Image.BILINEAR)

    # 雜訊
    if random.random() < 0.5:

```

```

arr = np.asarray(img, dtype=np.float32)
arr += np.random.normal(0, random.uniform(3, 15), arr.shape)
img = Image.fromarray(np.clip(arr, 0, 255).astype(np.uint8))
return img

```

想一想 透視變形用的 `perspective_coeffs()` 是在解一個 8 元一次聯立方程式：已知四個角變形前後的座標，求出能把新座標映射回原座標的 8 個係數。這正是 PIL `PERSPECTIVE` 需要的參數。

3.5 多行程平行產生

generate_dataset.py

```

def generate(out_dir, n, workers, seed, force=False):
    if os.path.exists(os.path.join(out_dir, "labels.csv")) and not force:
        print(f" {out_dir} 已存在，略過 (要重新產生請加 --force)")
        return
    os.makedirs(out_dir, exist_ok=True)
    chunk = max(1, n // (workers * 4))
    jobs = [(out_dir, s, min(chunk, n - s), seed + s) for s in range(0, n, chunk)]
    rows = []
    with Pool(workers) as pool:
        for i, part in enumerate(pool.imap(_worker, jobs), 1):
            rows.extend(part)
            print(f"\r {out_dir}: {len(rows)}/{n}", end="", flush=True)
    print()
    with open(os.path.join(out_dir, "labels.csv"), "w", newline="", encoding="utf-8") as f:
        w = csv.writer(f)
        w.writerow(["filename", "text"])
        w.writerows(rows)

```

把 n 張圖切成很多小塊，用 `multiprocessing.Pool` 讓多個 CPU 核心同時畫圖。每一塊用不同的 `seed` (亂數種子)，所以結果可重現：同樣的參數再跑一次，會得到一模一樣的資料集。驗證集的 `seed` 加了 10,000,000，確保和訓練集不重複。

4. 模型：plate_model.py

4.1 影像前處理與文字編碼

plate_model.py

```

def encode(text):
    return [CHARS.index(c) + 1 for c in text]

def preprocess(img):
    """PIL 影像 → (1, 32, 128) 的 tensor。"""
    img = img.convert("L").resize((IMG_W, IMG_H), Image.BILINEAR)
    arr = np.asarray(img, dtype=np.float32) / 255.0
    return torch.from_numpy((arr - 0.5) / 0.5).unsqueeze(0)

```

不管原圖多大，一律轉成灰階、縮放成 高 32 × 寬 128，再把像素值從 0–255 換成 -1~1。文字則轉成數字：`encode('AB-1')` 會得到每個字元在 `CHARS` 裡的位置 +1，因為編號 0 要保留給 CTC 的 blank。

4.2 CRNN 的結構

plate_model.py

```
def conv_block(cin, cout):
    return nn.Sequential(nn.Conv2d(cin, cout, 3, padding=1), nn.BatchNorm2d(cout), nn.ReLU(inplace=True))

class CRNN(nn.Module):
    def __init__(self, num_classes=len(CHARS) + 1):
        super().__init__()
        self.cnn = nn.Sequential(
            conv_block(1, 64), nn.MaxPool2d(2, 2),          # 16 x 64
            conv_block(64, 128), nn.MaxPool2d(2, 2),      # 8 x 32
            conv_block(128, 256), conv_block(256, 256),
            nn.MaxPool2d((2, 1), (2, 1)),                 # 4 x 32
            conv_block(256, 256), nn.MaxPool2d((4, 1), (4, 1)), # 1 x 32
            nn.Dropout2d(0.2),
        )
        self.rnn = nn.LSTM(256, 128, num_layers=2, bidirectional=True, dropout=0.2)
        self.fc = nn.Linear(256, num_classes)

    def forward(self, x):
        f = self.cnn(x).squeeze(2)          # (N, 256, T)
        f = f.permute(2, 0, 1)              # (T, N, 256)
        out, _ = self.rnn(f)
        return self.fc(out)                 # (T, N, C)
```

CRNN = Convolutional + Recurrent Neural Network。關鍵是 CNN 的池化設計：前兩次池化把高和寬都減半，之後只壓高度、不壓寬度，最後得到高度 1、寬度 32 的特徵圖。寬度方向的每一格，對應原圖由左到右的一個窄條，就像把車牌切成 32 個時間步，交給 LSTM 依序讀。



圖 3 一張圖在模型裡的形狀變化 (N = batch size)。藍色是 CNN，橘色是序列部分。

為什麼要用雙向 LSTM？判斷某一格是什麼字時，看看左右兩邊的內容很有幫助——例如一個筆畫被陰影遮住一半，鄰近幾格的資訊能幫忙補回來。整個模型約 235 萬個參數（訓練時會印出 模型參數量：2,351,524）。

5. CTC : 不用標位置也能學會讀字

模型對每張圖輸出 32 個時間步，但車牌只有 6–8 個字，而且我們不知道每個字落在第幾步。CTC (Connectionist Temporal Classification) 的解法是：多加一個特殊類別 **blank** (記作 $\emptyset$ ，編號 0)，並規定輸出要經過「合併連續重複 → 刪除 blank」才變成最終文字。

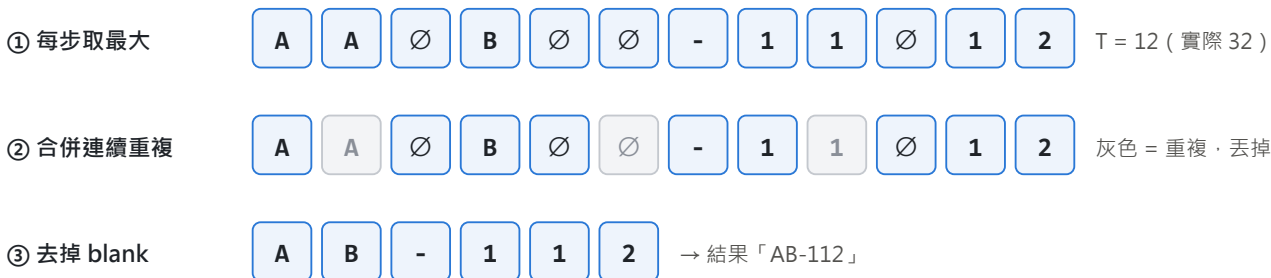


圖 4 CTC 解碼的例子。注意中間的 $\emptyset$ 讓兩個「1」得以保留成「11」。

訓練時，`nn.CTCLoss` 會把所有能解碼成正確答案的路徑 (例如「AAB」「A $\emptyset$ B」「 $\emptyset$ AB」...) 的機率加總，並讓這個總機率越大越好。模型因此自己學會字在哪裡，我們只需要提供整串答案。

5.1 推論 : greedy_decode()

plate_model.py

```
def greedy_decode(logits):
    """logits: (T, N, C) → [(text, confidence), ...]"""
    probs = logits.softmax(2).permute(1, 0, 2) # (N, T, C)
    best_p, best_i = probs.max(2)
    results = []
    for p_seq, i_seq in zip(best_p.tolist(), best_i.tolist()):
        text, conf, prev = [], 1.0, 0
        for p, i in zip(p_seq, i_seq):
            if i != 0 and i != prev:
                text.append(CHARS[i - 1])
                conf *= p
            prev = i
        results.append(("".join(text), conf if text else 0.0))
    return results
```

每個時間步取機率最高的類別 (greedy)，然後套用上面的規則：`i != 0` 去掉 blank、`i != prev` 合併重複。信心度是所有輸出字元機率的乘積，網頁上顯示的「信心度」就是它。

小提醒 CHARS 裡本來就有 - 這個字元，它是車牌上的連字號，和 blank 是兩回事。這也是 blank 不用「-」表示、而在圖 4 用 $\emptyset$ 的原因。

6. 訓練流程：train.py

6.1 準備

train.py (main 內)

```
model = CRNN().to(device)
progress.say(f"模型參數量：{sum(p.numel() for p in model.parameters()):,}")
optimizer = torch.optim.Adam(model.parameters(), lr=a.lr)
steps_per_epoch = (len(X_train) + a.batch - 1) // a.batch
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=a.lr, epochs=a.epochs, steps_per_epoch=steps_per_epoch
)
ctc = nn.CTCLoss(blank=0, zero_infinity=True)
```

元件	設定	作用
Adam	lr 預設 1e-3	根據梯度更新參數的演算法，對學習率不太敏感，很適合入門
OneCycleLR	max_lr = --lr	學習率先由小升到最大、再慢慢降到很小。前期跑得快、後期收得穩
CTCLoss	blank=0, zero_infinity=True	第 5 章的 CTC；zero_infinity 避免某些不可能的對齊產生無限大 loss

6.2 訓練迴圈

train.py (每個 batch)

```
for s in range(0, len(perm), a.batch):
    idx = perm[s:s + a.batch]
    x = train_augment(X_train[idx].to(device))
    tgt = [targets[i] for i in idx.tolist()]
    logits = model(x)
    log_probs = logits.log_softmax(2)
    loss = ctc(
        log_probs,
        torch.cat(tgt),
        torch.full((len(idx),), logits.size(0), dtype=torch.long),
        torch.tensor([len(t) for t in tgt], dtype=torch.long),
    )
    optimizer.zero_grad()
    loss.backward()
    nn.utils.clip_grad_norm_(model.parameters(), 5)
    optimizer.step()
    scheduler.step()
    total_loss += loss.item() * len(idx)
```

這就是 PyTorch 訓練的標準五步：前向 (`model(x)`) → 算 loss → `zero_grad()` 清掉舊梯度 → `backward()` 反向傳播 → `optimizer.step()` 更新參數。另外兩個細節：

- `clip_grad_norm(..., 5)`：梯度太大時等比例縮小，防止 LSTM 偶爾的梯度爆炸讓訓練崩掉。
- `CTCLoss` 需要四樣東西：log 機率、所有答案接成一長串、每張圖的時間步數（都是 32）、每個答案的長度。

每個 batch 送進模型前，還會再做一次即時資料增強：

train.py

```
def train_augment(x):
    """訓練時的即時資料增強：隨機亮度/對比與小幅平移。"""
    n = x.size(0)
    x = x * torch.empty(n, 1, 1, 1, device=x.device).uniform_(0.7, 1.3) \
        + torch.empty(n, 1, 1, 1, device=x.device).uniform_(-0.2, 0.2)
    shift = int(torch.randint(-3, 4, (1,)))
    return torch.roll(x, shifts=shift, dims=3).clamp(-1, 1)
```

同一張圖在不同 epoch 會有不同的亮度和位置，等於免費多出很多資料。

6.3 驗證：兩種正確率

train.py

```
def edit_distance(a, b):
    dp = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        prev, dp[0] = dp[0], i
        for j in range(1, len(b) + 1):
            prev, dp[j] = dp[j], min(dp[j] + 1, dp[j - 1] + 1, prev + (a[i - 1] != b[j - 1]))
    return dp[-1]

def evaluate(model, X, texts, device, batch=512):
    model.eval()
    preds = []
    with torch.no_grad():
        for i in range(0, len(X), batch):
            preds += [t for t, _ in greedy_decode(model(X[i:i + batch].to(device)))]
    seq_acc = sum(p == t for p, t in zip(preds, texts)) / len(texts)
    errors = sum(edit_distance(p, t) for p, t in zip(preds, texts))
    char_acc = 1 - errors / sum(len(t) for t in texts)
    return seq_acc, char_acc
```

指標	算法	例子：答案 ABC-1234，模型讀成 ABC-1284
整張正確率	完全一樣才算對	0% (這張算錯)
字元正確率	1 - 編輯距離總和 / 答案總字數	1 - 1/8 = 87.5% (只錯 1 個字)

編輯距離 (Levenshtein distance) 是「最少要改幾個字才會一樣」，包含替換、插入、刪除。實際應用看的是整張正確率——車牌錯一個字就是錯的車；字元正確率則能看出模型「差多少」。

6.4 你的真實訓練結果

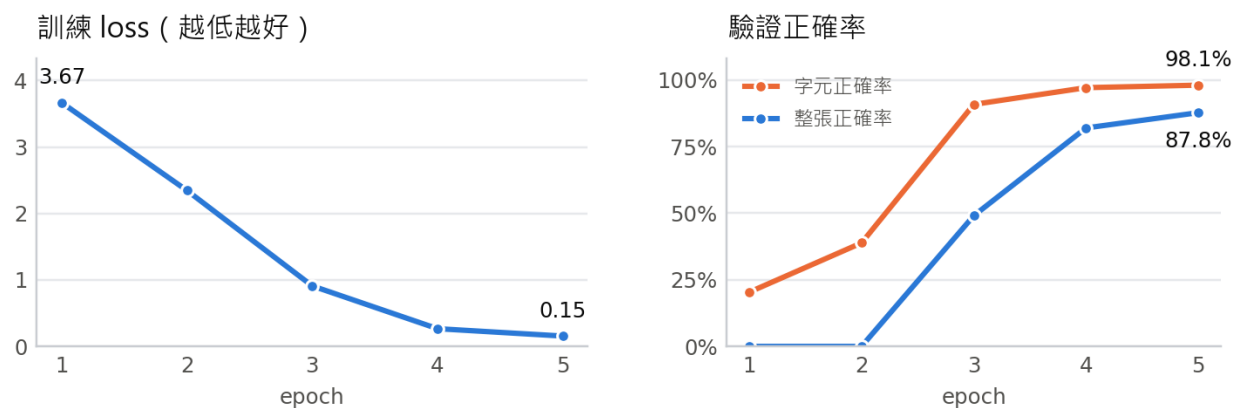


圖 5 models/runs/20261002-000605 : 5 個 epoch、batch 32、lr 0.001。

Epoch	訓練 loss	整張正確率	字元正確率
1	3.6669	0.00%	20.37%
2	2.3453	0.00%	38.86%
3	0.9093	49.10%	90.87%
4	0.2657	82.03%	97.16%
5	0.1548	87.77%	98.10%

前幾個 epoch 整張正確率常常是 0%，這很正常：CTC 模型一開始常只會輸出 blank 或少數幾個字，要先學會「字大概在哪」，整張正確率才會突然開始上升。只訓練 1 個 epoch 的模型幾乎全錯，原因就在這裡。

6.5 存檔與訓練進度

資料夾結構

```
models/
|-- active.txt          網站目前使用哪個模型
`-- runs/
    |-- 20261002-000605/ 每次訓練一個資料夾，不會互相覆蓋
        |-- plate_crnn.pt 驗證整張正確率最高那個 epoch 的權重
        |-- history.json   每個 epoch 的 loss / 正確率 (網頁畫曲線用)
        |-- config.json    訓練參數與最佳成績 (網頁「已儲存的模型」表格用)
        |-- progress.json 即時進度 (網頁進度條用，最多每秒更新一次)
```

每個 epoch 驗證完，只有在整張正確率創新高時才覆寫 `plate_crnn.pt`，所以中途按 Ctrl+C 停掉也不會失去最好的那一版。第一次存檔時還會改寫 `active.txt`，讓網站自動換成這次的新模型。

Progress 類別負責回報進度：同時寫 `progress.json` (給網頁的進度條)，並在終端機用 `\r` 覆寫同一行印出文字進度條：

終端機輸出

```
Epoch 2/5 [#####-----] 55% batch 520/938 預估剩餘 01:40
```

小提醒 寫 `progress.json` 時先寫到 `.tmp` 再用 `os.replace()` 換過去。這樣網站讀檔時，看到的永遠是完整的舊檔或新檔，不會讀到寫一半的內容。

注意 `.gitignore` 排除了 `models/` 和 `dataset/`，訓練好的模型只存在這台電腦，不會進 `git`。要備份或分享模型，請自行複製該次的 `run` 資料夾。

7. 網站：app.py 與前端

7.1 API 一覽

路由	方法	做什麼
<code>/</code>	GET	回傳網頁 (<code>index.html</code>)
<code>/api/status</code>	GET	資料集張數、目前使用的模型與成績
<code>/api/models</code>	GET	列出所有已儲存的模型
<code>/api/models/active</code>	POST	切換網站使用的模型 (改寫 <code>active.txt</code>)
<code>/api/recognize</code>	POST	上傳圖片 → 回傳辨識文字與信心度
<code>/api/random</code>	GET	即時產生一張合成車牌並辨識，回傳圖片 (<code>base64</code>)、答案、結果
<code>/api/val-samples</code>	GET	從驗證集隨機抽 <code>n</code> 張辨識
<code>/api/history</code>	GET	某次訓練的 <code>history.json</code>
<code>/api/progress</code>	GET	最近一次訓練的 <code>progress.json</code> ，加上「幾秒前更新」
<code>/api/jobs/generate</code>	POST	在背景執行 <code>generate_dataset.py</code>
<code>/api/jobs/train</code>	POST	在背景執行 <code>train.py</code>
<code>/api/jobs/stop</code>	POST	停止背景工作
<code>/api/jobs</code>	GET	背景工作狀態與最後 40 行執行紀錄

7.2 模型熱更新

app.py

```
def get_model():
    """載入目前使用的模型；切換模型或模型檔更新（例如訓練中）時重新載入。"""
    global _model, _model_key
    run = active_run()
    if run is None:
        return None
    path = run_file(run, "plate_crnn.pt")
    key = (run, os.path.getmtime(path))
    if key != _model_key:
        _model, _model_key = load_model(path), key
    return _model

def recognize(images):
    with lock:
        model = get_model()
        if model is None:
            return None
        with torch.no_grad():
            return greedy_decode(model(torch.stack([preprocess(im) for im in images])))
```

每次辨識前，用「模型名稱 + 檔案修改時間」當作 key。只要切換了模型、或訓練中 `plate_crnn.pt` 被更新，key 就會變，於是自動重新載入——所以訓練時開著網站，辨識結果會越來越準。`lock` 確保多個請求同時進來時，不會兩個人同時在載入模型。

7.3 從網頁啟動訓練

app.py

```
def start_job(name, args):
    if job_running():
        return jsonify(error=f"目前正在執行「{job['name']}」，請等它結束或先停止"), 409
    os.makedirs(os.path.dirname(LOG_PATH), exist_ok=True)
    log = open(LOG_PATH, "w", encoding="utf-8")
    env = dict(os.environ, PYTHONIOENCODING="utf-8")
    job["proc"] = subprocess.Popen(
        [sys.executable, "-u"] + args, cwd=BASE_DIR, stdout=log, stderr=subprocess.STDOUT, env=env
    )
    job["name"] = name
    return jsonify(ok=True)
```

網頁按「開始訓練」時，後端用 `subprocess.Popen` 另開一個 Python 行程執行 `train.py`，輸出寫進 `logs/job.log`。`-u` 讓輸出不經緩衝，網頁才能即時看到。同一時間只允許一個工作，避免兩個訓練搶 GPU。

前端 (`static/app.js`) 則是用輪詢：每 2 秒問一次 `/api/jobs` 和 `/api/progress` 更新執行紀錄與進度條，每 10 秒更新一次訓練曲線。超過 60 秒沒有新進度，進度條會變灰並顯示「訓練已停止或中斷」。

7.4 上傳與框選

上傳的照片畫在 `<canvas>` 上，拖曳滑鼠時記錄框選範圍（換算回原圖座標）。按「辨識」時，前端先把框選範圍裁切成新圖，再以 PNG 上傳到 `/api/recognize`——也就是說，裁切是在瀏覽器裡做的，後端收到的已經是車牌本身。

8. 動手實驗

理解程式碼最好的方法是改改看。每次訓練都會存成獨立資料夾，可以放心實驗，再到網頁上比較曲線：

實驗	怎麼做	觀察什麼
訓練久一點	<code>python train.py --epochs 30 --name e30</code>	正確率能到多高？什麼時候開始變平？
學習率	<code>--lr 3e-3 --name lr3e-3</code> 和 <code>--lr 3e-4</code>	太大會不穩定、太小學得慢
資料量	產生 5000 張 vs 50000 張（加 <code>--force</code> ）	資料越多越好嗎？差多少？
拿掉資料增強	把 <code>augment()</code> 改成只做縮放	驗證集分數變高，但真實照片變差——為什麼？
縮小模型	把 LSTM 改成 1 層、通道數減半	參數少很多，正確率掉多少？
真實照片	拍幾張真的車牌上傳測試	找出模型的弱點，回頭調整資料產生方式

目前的限制

- 只做文字辨識，需要先框選車牌。完整系統還需要一個「車牌偵測」模型（例如 YOLO）先找出車牌位置。
- 訓練資料全是合成的，和真實照片仍有差距：反光、髒污、螺絲遮擋、特殊字型、新式車牌的細節等。
- 驗證集也是合成的，所以高分只代表「很會讀合成車牌」，真實表現要用真實照片測。

常見問題

狀況	原因與解法
網頁看不到進度條或新功能	瀏覽器還在用舊版 app.js。按一次 Ctrl + F5 強制重新整理。
辨識結果幾乎全錯	模型訓練太少。只跑 1-2 個 epoch 時整張正確率通常是 0%，至少訓練 5 個以上。
網頁打不開	app.py 沒有在跑。開終端機執行 <code>python app.py</code> ，並保持視窗開著。
按「產生資料集」是灰色的	資料集已經存在。要重產請在終端機加 <code>--force</code> 。
進度條變灰、顯示中斷	超過 1 分鐘沒有新進度：訓練被停止、當掉，或電腦休眠了。看執行紀錄最後幾行找原因。

附錄：名詞對照

名詞	說明
epoch	把整個訓練集完整看過一遍
batch / batch size	一次送進模型的一小批圖片 / 這批有幾張
loss	模型答案和正確答案差多遠的數值，訓練就是讓它越來越小
learning rate (lr)	每次更新參數要走多大一步
CNN	卷積神經網路，擅長從圖片抽取邊緣、筆畫等局部特徵
LSTM	一種循環神經網路，擅長依序讀取序列，並記住前面看過的內容
CTC	讓序列輸出不需要逐字對齊標註的 loss 與解碼方法
blank	CTC 額外加的「這一步不輸出任何字」類別
資料增強	對訓練圖片做隨機變化，讓模型學到更通用的特徵
驗證集	訓練時不拿來學、只拿來打分數的資料，用來判斷模型有沒有真的學會
過擬合	模型把訓練資料背起來，對新資料卻表現不好
checkpoint	存下來的模型權重檔（這裡是 plate_crnn.pt）